Assignment 9: Advanced SQL Queries Using Joins and Subqueries

1. Introduction

Joins combine related rows from different tables. Subqueries place one query inside another and are useful for comparisons, filtering, and summary reports.

2. Report 1: booking details using INNER JOIN

```
SELECT b.booking_id, g.guest_name, r.room_number, r.room_type,
       b.check_in, b.check_out, b.booking_status
FROM Bookings b
INNER JOIN Guests g ON b.guest_id = g.guest_id
INNER JOIN Rooms r ON b.room_id = r.room_id
ORDER BY b.booking_id;
```

booking_id	guest_name	room_number	room_type	check_in	check_out	status
1	Aarav Sharma	102	Double	2026-10-01	2026-10-03	Checked-Out
2	Meera Nair	201	Deluxe	2026-10-04	2026-10-07	Checked-Out
3	Rahul Verma	101	Single	2026-10-10	2026-10-12	Checked-Out
4	Aarav Sharma	301	Double	2026-10-15	2026-10-18	Checked-In
5	Priya Iyer	302	Deluxe	2026-10-20	2026-10-22	Confirmed
6	Kabir Khan	102	Double	2026-10-23	2026-10-25	Confirmed
7	Meera Nair	101	Single	2026-09-20	2026-09-22	Checked-Out
8	Rahul Verma	201	Deluxe	2026-10-26	2026-10-28	Cancelled

Explanation: The join replaces guest_id and room_id with readable guest and room information and includes the dates selected by the query.

3. Report 2: payments and guest details

```
SELECT p.payment_id, g.guest_name, p.amount, p.payment_method, p.payment_date
FROM Payments p
JOIN Bookings b ON p.booking_id = b.booking_id
JOIN Guests g ON b.guest_id = g.guest_id
ORDER BY p.payment_id;
```

payment_id	guest_name	amount	payment_method	payment_date
1	Aarav Sharma	5600.00	UPI	2026-10-01
2	Meera Nair	12600.00	Card	2026-10-04
3	Rahul Verma	3600.00	Cash	2026-10-10
4	Aarav Sharma	9000.00	UPI	2026-10-15
5	Priya Iyer	9000.00	Bank Transfer	2026-10-20
6	Kabir Khan	5600.00	Card	2026-10-23
7	Meera Nair	3600.00	UPI	2026-09-20

Explanation: This report shows who made each payment, how much was paid, the method used, and the date.

4. Report 3: service usage and cost

```
SELECT bs.booking_id, s.service_name, bs.quantity,
       s.service_price, bs.quantity * s.service_price AS line_total
FROM Booking_Services bs
JOIN Services s ON bs.service_id = s.service_id
```

```
ORDER BY bs.booking_id, s.service_name;
```

booking_id	service_name	quantity	service_price	line_total
1	Breakfast	2	350.00	700.00
1	Laundry	1	250.00	250.00
2	Airport Pickup	1	1200.00	1200.00
2	Breakfast	3	350.00	1050.00
4	Breakfast	2	350.00	700.00
4	Laundry	2	250.00	500.00
5	Airport Pickup	1	1200.00	1200.00
6	Breakfast	2	350.00	700.00
7	Laundry	1	250.00	250.00

Explanation: Multiplying quantity by service price calculates the total cost for each service line.

5. Report 4: guests with bookings above the average booking count (subquery)

```
SELECT g.guest_id, g.guest_name, COUNT(b.booking_id) AS total_bookings
FROM Guests g
JOIN Bookings b ON g.guest_id = b.guest_id
GROUP BY g.guest_id, g.guest_name
HAVING COUNT(b.booking_id) > (
    SELECT AVG(booking_count)
    FROM (
        SELECT COUNT(*) AS booking_count
        FROM Bookings
        GROUP BY guest_id
    ) AS guest_booking_counts
);
```

guest_id	guest_name	total_bookings
1	Aarav Sharma	2
2	Meera Nair	2
3	Rahul Verma	2

Explanation: The subquery calculates the average number of bookings per guest represented in Bookings: 8 bookings divided across 5 guests is 1.6. Guests with two bookings are above that average.

6. Report 5: bookings with payment totals (LEFT JOIN)

```
SELECT b.booking_id, b.booking_status,
       COALESCE(SUM(p.amount), 0) AS total_paid
FROM Bookings b
LEFT JOIN Payments p ON b.booking_id = p.booking_id
GROUP BY b.booking_id, b.booking_status
ORDER BY b.booking_id;
```

booking_id	booking_status	total_paid
1	Checked-Out	5600.00
2	Checked-Out	12600.00
3	Checked-Out	3600.00
4	Checked-In	9000.00
5	Confirmed	9000.00
6	Confirmed	5600.00
7	Checked-Out	3600.00
8	Cancelled	0.00

Explanation: LEFT JOIN keeps every booking even if no payment exists. COALESCE displays zero instead of NULL for bookings without a payment.

7. Conclusion

The advanced queries produce useful hotel reports by combining guest, room, booking, payment, and service data. Joins make related details readable, while the subquery compares each guest's booking count with the average.